Blood Donation Manager

A full-stack application that centralises donor registration, blood requests and donation coordination in one secure platform.

Presenter · Puneet Goswami

Stack · React, FastAPI, PostgreSQL

Access · donor, requester, admin

Where coordination breaks down

01 **Donor records live in phone contacts**

Blood group and city sit in contacts, not in a record anyone can search.

02 **Availability is never current**

Nothing says whether a donor can be contacted this week or next.

03 **Request status lives in chat threads**

Pending, in progress and fulfilled are tracked across messages.

04 **Oversight and privacy pull apart**

Search must not expose phone numbers; admins still need the record.

WHO IT AFFECTS

Hospitals, blood banks, requesters coordinating urgent needs, and volunteer donors.

WHY A DIGITAL SOLUTION

Searchable records, role-based access, tracked request status and privacy-protected donor search.

Source: project brief and workflow walkthrough

The stack, layer by layer

LAYER	TECHNOLOGY	PURPOSE
Frontend	React 18 + Vite, React Router, React Hook Form, CSS	UI, routing, form validation, mobile-first CSS
Backend	FastAPI, Pydantic, SQLAlchemy	REST routes, schema validation, ORM queries
Database	PostgreSQL, Alembic	Relational store, versioned schema migrations
Security	JWT via python-jose, bcrypt	Stateless tokens, hashed passwords, role checks
Delivery	pytest, Docker Compose, GitHub Actions	API tests, containers, CI test and lint

Frontend on :5173, API on :8000, PostgreSQL in Docker Compose — one command brings the stack up.

Source: repository README and requirements files

What the application does

Authentication

Register, login and logout on JWT; session restored on refresh.

Role-based access

Donor, requester and administrator, enforced on every route.

Donor registration

Blood group, city, phone, availability and contact consent.

Donor search

Filter by group, city and availability; phone numbers hidden.

Blood requests

Create, view, update and cancel a request through its lifecycle.

Status tracking

Pending to in progress to fulfilled or cancelled, with history.

Coordination

Admin links a donor to a request and tracks contact status.

Dashboard

Role-scoped statistics for donors, requesters and admins.

Responsive UI

Checked at 600px, 800px and desktop widths.

Screenshots in README.md · login, register, dashboard, donor search, requests, Swagger

How a request moves through the system

03	**Frontend**	React 18 + Vite on :5173 · pages for login, register, dashboard, donors and requests · AuthContext restores the JWT from localStorage · axios injects the Bearer token	BROWSER
02	**Backend**	FastAPI on :8000 · /auth /donors /requests /coordinations /dashboard /admin over five services · JWT verification, role checks and ownership checks on every call	SERVER
01	**Database**	PostgreSQL · users, donors, blood_requests, donation_coordination, request_status_history · unique email, one donor per user, one coordination per request and donor pair	STORE

Data flow · user action → API call → FastAPI route → service layer → SQLAlchemy → PostgreSQL → response

72 checks, no failures

CATEGORY	PASSED	FAILED	NOT TESTED
Authentication	12	0	3
Donor profile	13	0	0
Blood requests	18	0	0
Coordination	8	0	0
Dashboard	4	0	0
Database / persistence	7	0	0
Frontend UI	10	0	0
Total	72	0	3

APPROACH

5 pytest API workflow tests

ESLint clean, 0 warnings

Swagger UI checks every endpoint

SECURITY VERIFIED

bcrypt hashing, never plain text

JWT expires after 60 minutes

.env files excluded from Git

Phone numbers hidden in search

Role checks on every endpoint

BUG FOUND AND FIXED

Donor dashboard showed total requests, now pending. No regressions.

Source: pytest run, ESLint output and Swagger UI walkthrough

The demo, in eight steps

01	**Register**	Create a donor account: name, email, password, role donor.
02	**Donor profile**	Add blood group, phone, city, availability and consent to contact.
03	**Log in as requester**	A separate account with role requester.
04	**Blood request**	Pick blood group, units, hospital, city and urgency, then submit.
05	**Search donors**	Filter by group and city; consenting donors listed, phones hidden.
06	**Admin coordination**	Log in as admin, link a donor to the request, set contact status.
07	**Track the request**	Status moves pending to in progress to fulfilled, with history.
08	**View the dashboard**	Role-scoped counts update as each step lands.

Donor and requester register in the UI · admin via python create_admin.py · API docs at localhost:8000/docs

Four problems that shaped the build

01 Role-based authorisation

Every service method re-checks role and ownership, not just the route.

02 Request status lifecycle

An allowed_transitions map rejects pending straight to fulfilled.

03 Donor privacy

DonorSearchOut drops contact fields; consent_to_contact filters rows.

04 Session persistence

AuthContext restores the token and re-verifies via /auth/me on mount.

WHAT CARRIED OVER

Backend authorisation is the source of truth

Frontend role-gating is UX only

Pydantic gives validation and clear errors

Alembic makes schema changes versioned

Tests first catch duplicates and bad transitions

Routes to services to models keeps it readable

Source: implementation notes from the build

Questions welcome

Authentication, role-based access, donor management, request tracking and admin coordination, in one application.

COMPLETED

Donor, requester and admin roles with JWT auth
Donor profiles, search and consent flags
Request lifecycle with status history
Admin coordination and role-scoped dashboards
5 backend tests, ESLint clean, compile clean

NEXT

Email notifications on status change
Donor accept or decline on coordination
Request auto-expiry
Hospital directory and donation history
Distance-based search and admin analytics

TRY IT · LOCALHOST:5173 · API DOCS · LOCALHOST:8000/DOCS